Namespace

1 What is a Namespace?

In C++, **namespaces** are used to organize code and prevent **name conflicts** — especially in large projects or when using multiple libraries. A **namespace** groups related classes, functions, variables, etc., under a **name**.

Example:

```
#include <iostream>

namespace MathUtils {
    int add(int a, int b) {
        return a + b;
    }

    int square(int x) {
        return x * x;
    }
}

int main() {
    // Access using scope resolution operator ::
    std::cout << MathUtils::add(3, 5) << std::endl;
    std::cout << MathUtils::square(4) << std::endl;
    return 0;
}
```

Output:

```
8
16
```

2 Using `using` Declarations

If you use a namespace often, you can simplify access with `using`:

Option 1 — Use a single name:

```
#include <iostream>
using MathUtils::add;

int main() {
    std::cout << add(2, 3) << std::endl;
    std::cout << square(4);
}
```

Option 2 — Use the entire namespace:

```
#include <iostream>
using namespace MathUtils;

int main() {
    std::cout << add(2, 3) << std::endl;
    std::cout << square(4) << std::endl;
}
```

Be careful with `using namespace` in headers or large projects — it may cause **name collisions**.

3 Nested Namespaces

C++ allows **nested namespaces** for hierarchical organization.

```
#include <iostream>
namespace Company {
    namespace Project {
        void info() {
```

```
            std::cout << "Project info\n";
        }
    }
}

// C++17 and later allows this shorthand:
namespace Company::Project {
    void details() {
        std::cout << "Project details\n";
    }
}

int main() {
    Company::Project::info();
    Company::Project::details();
}
```

4 Anonymous Namespaces

Used to **limit visibility** of functions or variables to a single translation unit (file).

```
#include <iostream>
namespace {
    int secret = 42;

    void hiddenFunction() {
        std::cout << "Hidden value: " << secret << std::endl;
    }
}

int main() {
    hiddenFunction();  // OK
    // secret accessible only in this file
}
```

Equivalent to using `static` in C for file-local scope.

5 Namespace Aliases

You can create **shorter aliases** for long namespace names.

```cpp
#include <iostream>
namespace VeryLongNamespaceName {
    void greet() {
        std::cout << "Hello!\n";
    }
}

namespace VLN = VeryLongNamespaceName;

int main() {
    VLN::greet();  // Easier to read
}
```

6 Extending Namespaces

You can **split** namespace definitions across multiple files or locations.

```cpp
// module1.cpp
namespace App {
    void start() { std::cout << "App started\n"; }
}

// module2.cpp
namespace App {
    void stop() { std::cout << "App stopped\n"; }
}

// main.cpp
int main() {
    App::start();
    App::stop();
}
```

Summary

CONCEPT	SYNTAX	USE CASE
Basic namespace	`namespace N { ... }`	Organize code

CONCEPT	SYNTAX	USE CASE
Access member	`N::member`	Qualified access
Using declaration	`using N::x;`	Simplify usage
Using directive	`using namespace N;`	Temporarily bring all names into scope
Nested	`namespace A::B {}`	Hierarchical structure
Anonymous	`namespace {}`	File-local visibility
Alias	`namespace short = long_name;`	Shorter references